LANGUAGES AND AUTOMATA I

HOMEWORK #1 PARTIAL ANSWERS

exercises.lua

```lua
function change(amount)
  if math.type(amount) ~= "integer" then
    error("Amount must be an integer")
  end
  if amount < 0 then
    error("Amount cannot be negative")
  end
  local counts, remaining = {}, amount
  for _, denomination in ipairs({25, 10, 5, 1}) do
    counts[denomination] = remaining // denomination
    remaining = remaining % denomination
  end
  return counts
end

function first_then_lower_case(array, predicate)
  local first = nil
  for _, s in ipairs(array) do
    if predicate(s) then
      return string.lower(s)
    end
  end
  return nil
end

function powers_generator(base, limit)
  local power = 1
  return coroutine.create(function()
    while power <= limit do
      coroutine.yield(power)
      power = power * base
    end
  end)
```

```
end

function say(word)
  if word == nil then
    return ""
  end
  return function(next)
    if next == nil then
      return word
    else
      return say(word .. " " .. next)
    end
  end
end

function meaningful_line_count(filename)
  local count = 0
  local file = io.open(filename, 'r')
  if file == nil then
    error("No such file")
  end
  for line in file:lines() do
    line = line:match "^%s*(.*)"
    if line ~= "" and line:sub(1, 1) ~= "#" then
        count = count + 1
    end
  end
  file:close()
  return count
end

Quaternion = (function (class)
  local meta = {
    __add = function(self, q)
      return class.new(self.a + q.a, self.b + q.b, self.c + q.c, self.d + q.d)
    end,
    __mul = function(self, q)
      return class.new(
        q.a * self.a - q.b * self.b - q.c * self.c - q.d * self.d,
        q.a * self.b + q.b * self.a - q.c * self.d + q.d * self.c,
        q.a * self.c + q.b * self.d + q.c * self.a - q.d * self.b,
        q.a * self.d - q.b * self.c + q.c * self.b + q.d * self.a
      )
    end,
    __eq = function(self, q)
      return self.a == q.a and self.b == q.b and self.c == q.c and self.d == q.d
```

```
    end,
    __tostring = function(self)
      local s = ""
      for i, c in ipairs(self:coefficients()) do
        if c ~= 0 then
          s = s .. (c < 0 and "-" or s == "" and "" or "+")
          s = s .. ((i ~= 1 and math.abs(c) == 1) and "" or tostring(math.abs(c)))
          s = s .. ({"", "i", "j", "k"})[i]
        end
      end
      return s == "" and "0" or s
    end,
    __index = {
      coefficients = function(self)
        return {self.a, self.b, self.c, self.d}
      end,
      conjugate = function(self)
        return class.new(self.a, -self.b, -self.c, -self.d)
      end
    },
  }
  class.new = function (a, b, c, d)
    return setmetatable({a = a, b = b, c = c, d = d}, meta)
  end
  return class
end)({})
```

exercises.js

```
import { open } from "node:fs/promises"

export function change(amount) {
  if (!Number.isInteger(amount)) {
    throw new TypeError("Amount must be an integer")
  }
  if (amount < 0) {
    throw new RangeError("Amount cannot be negative")
  }
  let [counts, remaining] = [{}, amount]
  for (const denomination of [25, 10, 5, 1]) {
    counts[denomination] = Math.floor(remaining / denomination)
    remaining %= denomination
  }
  return counts
}
```

```
export function firstThenLowerCase(strings, predicate) {
  return strings.find(predicate)?.toLowerCase()
}

export function* powersGenerator({ ofBase: base, upTo: limit }) {
  for (let power = 1; power <= limit; power *= base) {
    yield power
  }
}

export function say(first) {
  if (first === undefined) {
    return ""
  }
  return (second) => {
    if (second === undefined) {
      return first
    }
    return say(`${first} ${second}`)
  }
}

export async function meaningfulLineCount(filename) {
  let count = 0
  const file = await open(filename, "r")
  for await (const line of file.readLines()) {
    // Note that readLines will autoclose the file, yay
    const trimmed = line.trim()
    if (trimmed && !trimmed.startsWith("#")) {
      count++
    }
  }
  return count
}

export class Quaternion {
  constructor(a, b, c, d) {
    Object.assign(this, { a, b, c, d })
    return Object.freeze(this)
  }
  plus(q) {
    return new Quaternion(
      this.a + q.a,
      this.b + q.b,
      this.c + q.c,
      this.d + q.d
```

```
    )
  }
  times(q) {
    return new Quaternion(
      q.a * this.a - q.b * this.b - q.c * this.c - q.d * this.d,
      q.a * this.b + q.b * this.a - q.c * this.d + q.d * this.c,
      q.a * this.c + q.b * this.d + q.c * this.a - q.d * this.b,
      q.a * this.d - q.b * this.c + q.c * this.b + q.d * this.a
    )
  }
  equals(q) {
    return this.a === q.a && this.b === q.b && this.c === q.c && this.d === q.d
  }
  get conjugate() {
    return new Quaternion(this.a, -this.b, -this.c, -this.d)
  }
  get coefficients() {
    return [this.a, this.b, this.c, this.d]
  }
  toString() {
    let s = ""
    for (const [i, c] of this.coefficients.entries()) {
      if (c === 0) continue
      s += c < 0 ? "-" : s == "" ? "" : "+"
      if (Math.abs(c) !== 1 || i === 0) s += Math.abs(c)
      s += ["", "i", "j", "k"][i]
    }
    return s || "0"
  }
}
```

exercises.py

```
from dataclasses import dataclass
from collections.abc import Callable


def change(amount: int) -> dict[int, int]:
    if not isinstance(amount, int):
        raise TypeError('Amount must be an integer')
    if amount < 0:
        raise ValueError('Amount cannot be negative')
    counts, remaining = {}, amount
    for denomination in (25, 10, 5, 1):
        counts[denomination], remaining = divmod(remaining, denomination)
    return counts
```

```
def first_then_lower_case(s: list[str], p: Callable[[str],bool], /) -> str:
    first = next((word for word in s if p(word)), '')
    return first.lower() if first else None


def powers_generator(*, base: int, limit: int):
    power = 1
    while power <= limit:
        yield power
        power *= base


def say(word: str|None=None, /):
    if word is None:
        return ''
    return lambda next=None: word if next is None else say(f'{word} {next}')


def meaningful_line_count(filename: str, /) -> int:
    count = 0
    with open(filename, 'r', encoding='utf-8') as file:
        for line in file:
            line = line.strip()
            if line and not line.startswith('#'):
                count += 1
    return count


@dataclass(frozen=True)
class Quaternion:
    a: float
    b: float
    c: float
    d: float
    def __add__(self, q: 'Quaternion') -> 'Quaternion':
        return Quaternion(self.a + q.a, self.b + q.b, self.c + q.c, self.d + q.d)
    def __mul__(self, q: 'Quaternion') -> 'Quaternion':
        return Quaternion(
            q.a * self.a - q.b * self.b - q.c * self.c - q.d * self.d,
            q.a * self.b + q.b * self.a - q.c * self.d + q.d * self.c,
            q.a * self.c + q.b * self.d + q.c * self.a - q.d * self.b,
            q.a * self.d - q.b * self.c + q.c * self.b + q.d * self.a)
    @property
    def coefficients(self) -> tuple[float, float, float, float]:
```

```
    return (self.a, self.b, self.c, self.d)
@property
def conjugate(self) -> 'Quaternion':
    return Quaternion(self.a, -self.b, -self.c, -self.d)
def __repr__(self) -> str:
    s = ""
    for c, unit in zip(self.coefficients, ["", "i", "j", "k"]):
        if c == 0: continue
        s += '-' if c < 0 else '' if s == '' else '+'
        s += '' if abs(c) == 1 and unit != "" else str(abs(c))
        s += unit
    return '0' if s == '' else s
```